

Литературный обзор: Современные методы прайсинга и хеджирования деривативов с помощью глубокого обучения

Бирюков Никита

14 мая 2026 г.

Содержание

1	Введение: Смена парадигмы в количественных финансах	3
1.1	Ведущие исследовательские группы и их вклад	3
1.1.1	Группа Deep BSDE (Принстон, ETH Zurich)	3
1.1.2	Группа DGM (University of Oxford, UIUC)	4
1.1.3	Группа Deep Hedging (JP Morgan, ETH Zurich)	4
1.1.4	Группа Deep Calibration (King's College London, Imperial College) .	5
1.1.5	Группа прикладных исследований (TU Delft, CWI, NYU)	5
2	Архитектуры нейронных сетей в финансовом моделировании	6
2.1	Полносвязные сети (Feed-Forward Networks / MLP)	6
2.1.1	Теоретическая справка	6
2.1.2	Применение в финансах	6
2.1.3	Преимущества и недостатки	6
2.2	Остаточные сети (Residual Networks, ResNets)	7
2.2.1	Теоретическая справка	7
2.2.2	Применение в финансах	7
2.2.3	Преимущества и недостатки	7
2.3	Рекуррентные сети (RNN / LSTM)	7
2.3.1	Теоретическая справка	7
2.3.2	Применение в финансах	7
2.3.3	Преимущества и недостатки	8
2.4	Сравнительный анализ архитектур	8
3	Детальный анализ ключевых исследований	8
3.1	Метод Deep BSDE: Стохастическая оптимизация для высокоразмерных PDE	8
3.1.1	Проблема и мотивация	8
3.1.2	Математическая теория	9
3.1.3	Алгоритм и применение нейросетей	9
3.1.4	Область применения и преимущества	10
3.2	Метод DGM: Глобальное решение и задачи со свободной границей	10
3.2.1	Проблема и мотивация	10
3.2.2	Математическая теория	10
3.2.3	Алгоритм и Архитектура	11

3.2.4	Область применения и преимущества	11
4	Детальный анализ ключевых исследований	12
4.1	Метод Deep BSDE: Стохастическая оптимизация для высокоразмерных PDE	12
4.1.1	Проблема и мотивация	12
4.1.2	Математическая теория	12
4.1.3	Алгоритм и применимость	12
4.2	DGM: Бессеточный метод Галёркина	13
4.2.1	Проблема и мотивация	13
4.2.2	Математическая теория	13
4.2.3	Алгоритм и применимость	13
4.3	Deep Hedging: Оптимизация торговых стратегий при наличии трений . .	13
4.3.1	Проблема и мотивация	13
4.3.2	Математическая теория	13
4.3.3	Алгоритм и применимость	14
4.4	Deep Calibration: Ускорение калибровки сложных моделей	14
4.4.1	Проблема и мотивация	14
4.4.2	Суть метода	14
4.4.3	Алгоритм и применимость	14
4.5	CaNN: Калибровка через механизм обратного распространения ошибки .	14
4.5.1	Суть метода	14
4.5.2	Применимость	15
4.6	Теоретическое обоснование: Рандомизированные сигнатуры	15
4.6.1	Суть метода	15
4.6.2	Применимость	15
4.7	Инженерные аспекты: Обеспечение финансовой корректности	15
4.7.1	Проблема и решения	15
4.7.2	Применимость	15
5	Сравнительный анализ и Выводы	16
5.1	План практического исследования	16

1 Введение: Смена парадигмы в количественных финансах

Классическая теория оценки производных финансовых инструментов (деривативов), фундаментально обоснованная работами Блэка, Шоулза и Мертона (1973) [2], опирается на стохастическое исчисление и решение уравнений в частных производных (PDE). Традиционные численные методы, такие как метод Монте-Карло (Monte Carlo) и конечно-разностные методы (Finite Difference Methods, FDM), являются «золотым стандартом» для задач низкой размерности. Однако современная финансовая индустрия сталкивается с вызовами, которые делают классические подходы вычислительно неэффективными:

1. **«Проклятие размерности» (Curse of Dimensionality):** Сложность сеточных методов растет экспоненциально ($O(n^d)$). Оценка опциона на корзину из 50–100 акций или расчет CVA (Credit Valuation Adjustment) для портфеля банка классическими методами требует неприемлемых временных затрат.
2. **Сложность калибровки (Calibration Bottleneck):** Современные модели, такие как модели грубой волатильности (Rough Volatility), лучше описывают рынок, но не имеют аналитических решений. Их калибровка через Монте-Карло занимает часы, что делает их неприменимыми для реал-тайм трейдинга.
3. **Рыночные трения:** Классические модели предполагают идеальный рынок. Учет транзакционных издержек и влияния на рынок (market impact) требует перехода к задачам стохастического управления, которые крайне сложны для аналитического решения.

В ответ на эти вызовы сформировалось направление *Scientific Machine Learning*, предлагающее использование глубоких нейронных сетей (DNN) как универсальных аппроксиматоров для решения финансовых задач.

1.1 Ведущие исследовательские группы и их вклад

Анализ современной литературы позволяет выделить несколько ключевых центров компетенций, каждый из которых специализируется на своем классе задач.

1.1.1 Группа Deep BSDE (Принстон, ETH Zurich)

Ключевые исследователи: Weinan E, Jiequn Han, Arnulf Jentzen.

- **Направление исследований:** Решение высокоразмерных параболических PDE (уравнение теплопроводности, Блэка-Шоулза, Аллена-Кана) с помощью нейронных сетей.
- **Достижения:**
 - Разработали метод *Deep BSDE* [5], основанный на стохастической формулировке уравнений (Backward Stochastic Differential Equations).
 - Впервые успешно решили нелинейное уравнение Блэка-Шоулза с риском дефолта для размерности $d = 100$ за разумное время.
 - Доказали (теоретически и эмпирически), что нейросети могут преодолевать проклятие размерности, аппроксимируя градиент решения без построения пространственной сетки.

- **Оборудование:** В оригинальной статье использовались кластеры с GPU (NVIDIA Tesla K80/P100). Обучение модели для $d = 100$ занимало порядка 30–40 минут.
- **Open Source / Код:**
 - Официальный репозиторий: <https://github.com/frankhan91/DeepBSDE> (TensorFlow)
 - Существует множество современных портов на PyTorch.
- **Ключевые публикации:** [5], [1].

1.1.2 Группа DGM (University of Oxford, UIUC)

Ключевые исследователи: Justin Sirignano, Konstantinos Spiliopoulos.

- **Направление исследований:** Бессеточные методы (Meshfree methods) для решения PDE со свободной границей (Free Boundary Problems), что критически важно для оценки Американских опционов.
- **Достижения:**
 - Разработали метод *DGM (Deep Galerkin Method)* [9], который минимизирует невязку дифференциального оператора в случайно выбранных точках пространства.
 - Успешно решили задачу прайсинга Американских опционов в размерности до $d = 200$, что невозможно для традиционных методов.
 - Предложили метод Монте-Карло для оценки вторых производных (Гессияна), снижающий вычислительную сложность с $O(d^2)$ до $O(d)$.
- **Оборудование:** Использовали суперкомпьютер *Blue Waters* (NCSA) с тысячами узлов GPU, так как метод DGM требует значительных вычислительных ресурсов для сходимости в высоких размерностях.
- **Open Source / Код:**
 - Реализации доступны в библиотеке *DeepXDE*: <https://github.com/lululxvi/deepxde>.
- **Ключевые публикации:** [9].

1.1.3 Группа Deep Hedging (JP Morgan, ETH Zurich)

Ключевые исследователи: Hans Buehler, Lukas Gonon, Josef Teichmann, Ben Wood.

- **Направление исследований:** Прямая оптимизация торговых стратегий хеджирования в условиях рыночных трений (транзакционные издержки, ликвидность).
- **Достижения:**
 - Сформулировали задачу хеджирования не как поиск «справедливой цены», а как минимизацию выпуклой меры риска (Convex Risk Measure) [3].
 - Показали, что нейросети (LSTM/RNN) способны самостоятельно выучивать сложные стратегии (например, «wait-and-see» или бандинг), которые экономически эффективнее классической Дельта-стратегии при наличии комиссий.

- Внедрили данную технологию в реальные бизнес-процессы банка JP Morgan.
- **Оборудование:** Для экспериментов использовались стандартные GPU (например, NVIDIA Tesla V100). Обучение на исторических данных занимает часы, но инференс происходит мгновенно.
- **Open Source / Код:**
 - Библиотека PFNet: <https://github.com/pfnet-research/deep-hedging>.
- **Ключевые публикации:** [3], [4] (теоретическое обоснование).

1.1.4 Группа Deep Calibration (King's College London, Imperial College)

Ключевые исследователи: Blanka Horvath, Christian Bayer, Aitor Muguruza.

- **Направление исследований:** Ускорение калибровки сложных стохастических моделей, в частности моделей грубой волатильности (Rough Volatility models: rBergomi, rHeston).
- **Достижения:**
 - Предложили двухэтапный подход: обучение нейросети-суррогата на синтетических данных (offline) и использование её для мгновенной калибровки (online) [Bayer2019, 6].
 - Достигли ускорения процесса калибровки в 10 000 – 60 000 раз по сравнению с традиционными методами, сохраняя точность на уровне метода Монте-Карло.
- **Оборудование:** Обучение проводилось на стандартных GPU (Google Colab, NVIDIA GTX 1080). Генерация обучающей выборки является самой ресурсоемкой частью (требует CPU-кластера или времени), само обучение сети происходит быстро.
- **Open Source / Код:**
 - Репозиторий авторов: <https://github.com/amuguruza/NN-StochVol-Calibrations>. Содержит готовые ноутбуки для моделей Heston и Rough Bergomi.
- **Ключевые публикации:** [6], [Bayer2019].

1.1.5 Группа прикладных исследований (TU Delft, CWI, NYU)

Ключевые исследователи: Cornelis Oosterlee, Shuaiqiang Liu (CaNN); Andrey Itkin.

- **Направление исследований:** Инженерная оптимизация нейросетевых солверов, обеспечение отсутствия арбитража, альтернативные методы калибровки.
- **Достижения:**
 - Разработали фреймворк *CaNN (Calibration Neural Network)* [8], где калибровка параметров модели выполняется через обратное распространение ошибки (Backpropagation) на входной слой сети, без внешнего оптимизатора.

- Andrey Itkin предложил методы регуляризации (Soft Constraints) для устранения статического арбитража в предсказаниях нейросети и обосновал необходимость использования гладких функций активации (Softplus, ELU) для корректного расчета «Греков» [7].
- **Open Source / Код:**
 - Примеры реализации CaNN доступны в сообществе (поиск по ключевым словам *Neural Network Calibration*).
- **Ключевые публикации:** [8], [7].

2 Архитектуры нейронных сетей в финансовом моделировании

Выбор архитектуры нейронной сети является критическим этапом в задачах количественных финансов. В отличие от задач компьютерного зрения или NLP, где существуют устоявшиеся стандарты (CNN, Transformers), в финансовом моделировании архитектура диктуется математической структурой задачи: марковостью процесса, зависимостью от траектории (path-dependency) и требованиями к гладкости производных.

Ниже приведен детальный анализ трех основных классов архитектур, применяемых в прайсинге и хеджировании.

2.1 Полносвязные сети (Feed-Forward Networks / MLP)

2.1.1 Теоретическая справка

Полносвязная сеть (Multilayer Perceptron, MLP) — это базовая архитектура, где каждый нейрон слоя l соединен со всеми нейронами слоя $l + 1$. Математически выход слоя описывается как:

$$h^{(l+1)} = \sigma(W^{(l)}h^{(l)} + b^{(l)}), \quad (1)$$

где W — матрица весов, b — вектор смещения, σ — нелинейная функция активации. Теоретическим обоснованием использования MLP служит *Универсальная теорема аппроксимации* (Hornik et al., 1989), утверждающая, что MLP с одним скрытым слоем может аппроксимировать любую непрерывную функцию с заданной точностью.

2.1.2 Применение в финансах

- **Deep Calibration:** Используется как суррогатная модель для отображения «Параметры модели $\rightarrow$ Цена опциона» или «Параметры $\rightarrow$ Поверхность волатильности» [6].
- **Inverse Problems:** Решение обратных задач, где по цене нужно восстановить параметры (например, локальную волатильность).

2.1.3 Преимущества и недостатки

Плюсы: Простота реализации; высокая скорость инференса; хорошо изученные методы регуляризации.

Минусы: Плохо работают с временными рядами и зависимостями от траектории; подвержены затуханию градиента при большом количестве слоев.

2.2 Остаточные сети (Residual Networks, ResNets)

2.2.1 Теоретическая справка

ResNets решают проблему затухания градиента в глубоких сетях путем введения «скип-соединений» (skip connections). Вместо того чтобы учить полное отображение $H(x)$, сеть учит остаточную функцию $F(x) = H(x) - x$. Слой ResNet выглядит так:

$$x_{l+1} = \sigma(W_l x_l + b_l) + x_l \quad (2)$$

Это позволяет сигналу беспрепятственно проходить через сотни слоев, что критически важно для аппроксимации сложных функционалов.

2.2.2 Применение в финансах

- **Решение PDE (DGM, Deep BSDE):** Методы DGM [9] и Deep BSDE [5] требуют очень глубоких сетей (до 50 слоев) для точной аппроксимации градиента решения в высокой размерности. Без ResNet-блоков такие сети практически невозможно обучить.

2.2.3 Преимущества и недостатки

Плюсы: Позволяют строить очень глубокие модели; эффективно обучаются; обеспечивают высокую точность аппроксимации сложных поверхностей.

Минусы: Сложнее в настройке гиперпараметров; требуют больше памяти при обучении по сравнению с простым MLP.

2.3 Рекуррентные сети (RNN / LSTM)

2.3.1 Теоретическая справка

Рекуррентные сети предназначены для обработки последовательностей. В отличие от FNN, они имеют «внутреннюю память» (hidden state h_t), которая передается от шага к шагу:

$$h_t = \sigma(W_x x_t + W_h h_{t-1} + b). \quad (3)$$

Архитектура **LSTM** (Long Short-Term Memory) добавляет механизм «ворот» (gates), позволяющий сети учить долгосрочные зависимости и избегать взрыва градиента.

2.3.2 Применение в финансах

- **Deep Hedging:** В задачах с транзакционными издержками оптимальное действие сегодня зависит от портфеля вчера. Это классическая марковская зависимость, идеально моделируемая LSTM [3].
- **Path-dependent опционы:** Оценка Азиатских опционов или Lookback-опционов, где выплата зависит от всей траектории цены.
- **Прогнозирование временных рядов:** Предсказание реализованной волатильности.

2.3.3 Преимущества и недостатки

Плюсы: Естественным образом работают с временными рядами; учитывают историю процесса (memory effect); необходимы для моделей с трениями.

Минусы: Медленное обучение (плохо распараллеливаются во времени); сложны в интерпретации; требуют больших вычислительных ресурсов (VRAM).

2.4 Сравнительный анализ архитектур

В Таблице 1 представлено сравнение архитектур в контексте задач дипломной работы.

Таблица 1: Сравнение архитектур нейронных сетей для финансового моделирования

Архитектура	Ключевая особенность	Область применения	Сложность обучения
FNN (MLP)	Универсальная аппроксимация, простота	Deep Calibration, простые суррогаты	Низкая (CPU/GPU)
ResNet	Skip connections, большая глубина	Решение PDE (DGM, Deep BSDE), высокая размерность	Средняя/Высокая (GPU)
LSTM/GRU	Память, обработка последовательностей	Deep Hedging, Path-dependent опционы	Высокая (требует мощных GPU)

Вывод: Для задач калибровки (Deep Calibration), где требуется мгновенное отображение параметров в цену, оптимальным выбором является FNN. Однако для задач хеджирования (Deep Hedging) и решения PDE в высокой размерности необходимо использование более сложных архитектур типа LSTM и ResNet соответственно, так как они способны улавливать временные зависимости и сложную геометрию решений.

3 Детальный анализ ключевых исследований

В данном разделе проводится глубокий анализ фундаментальных работ, формирующих методологическую базу исследования.

3.1 Метод Deep BSDE: Стохастическая оптимизация для высокоразмерных PDE

Статья: *Solving high-dimensional partial differential equations using deep learning* [5].

3.1.1 Проблема и мотивация

Классические сеточные методы (Finite Difference Methods) страдают от «проклятия размерности»: количество узлов сетки растет как N^d , где d — количество активов. Это делает невозможным их применение для $d > 5$. С другой стороны, стандартный метод Монте-Карло, хотя и работает в высоких размерностях, применим только для *линейных* PDE (через формулу Фейнмана-Каца).

Однако многие современные финансовые задачи (например, оценка CVA, прайсинг с учетом дефолтного риска или различных ставок заимствования) описываются **нелинейными** PDE. Метод Deep BSDE, предложенный группой исследователей из Принсто-

на и ETH Zurich (Weinan E, J. Han, A. Jentzen), стал первым алгоритмом, позволившим решать такие уравнения в размерности $d = 100$ за полиномиальное время.

3.1.2 Математическая теория

Метод базируется на теории обратных стохастических дифференциальных уравнений (Backward Stochastic Differential Equations, BSDE). Рассмотрим полулинейное параболическое уравнение для функции цены $u(t, x)$:

$$\frac{\partial u}{\partial t} + \frac{1}{2} \text{Tr}(\sigma \sigma^T \text{Hess}_x u) + \nabla u \cdot \mu + f(t, x, u, \sigma^T \nabla u) = 0, \quad (4)$$

с терминальным условием $u(T, x) = g(x)$. Здесь $f(\cdot)$ — нелинейная функция, описывающая, например, разницу процентных ставок или риск контрагента.

Согласно нелинейной версии формулы Фейнмана-Каца (теорема Парду-Пенга), решение этого PDE эквивалентно решению системы стохастических уравнений:

1. Прямое уравнение (Forward SDE) — Динамика активов:

$$X_t = \xi + \int_0^t \mu(s, X_s) ds + \int_0^t \sigma(s, X_s) dW_s \quad (5)$$

2. Обратное уравнение (Backward SDE) — Динамика портфеля:

$$Y_t = g(X_T) - \int_t^T f(s, X_s, Y_s, Z_s) ds - \int_t^T Z_s^T dW_s \quad (6)$$

Финансовый смысл переменных:

- $Y_t = u(t, X_t)$ — цена дериватива (или стоимость портфеля) в момент t .
- $Z_t = \sigma^T(t, X_t) \nabla u(t, X_t)$ — вектор хеджирования (аналог «Дельты», умноженной на волатильность), показывающий, сколько риска нужно перекрыть.

3.1.3 Алгоритм и применение нейросетей

В классических численных методах Z_t неизвестно. Идея Deep BSDE заключается в аппроксимации неизвестной функции градиента Z_t нейронной сетью на каждом временном шаге.

Алгоритм дискретизируется по времени $0 = t_0 < t_1 < \dots < t_N = T$ (схема Эйлера-Маруямы):

$$Y_{n+1} \approx Y_n - f(t_n, X_n, Y_n, Z_n) \Delta t + Z_n^T \Delta W_n \quad (7)$$

Здесь $Z_n = \text{NeuralNet}(X_n; \theta_n)$. Процесс начинается с Y_0 (искомая цена, обучаемый параметр) и идет вперед во времени.

Функция потерь (Loss Function) строится на невязке терминального условия:

$$L(\theta) = \mathbb{E} [|Y_T(\theta) - g(X_T)|^2] \quad (8)$$

Минимизируя разницу между тем, что мы «наsimулировали» (Y_T), и тем, что должны выплатить по контракту ($g(X_T)$), сеть одновременно находит справедливую цену Y_0 и стратегию хеджирования $\{Z_t\}$.

3.1.4 Область применения и преимущества

- **Где используется:** Основная ниша метода — задачи оценки кредитных рисков (**XVA: CVA/DVA/FVA**) для крупных портфелей банков, а также прайсинг опционов на корзины акций (Basket Options) в условиях нелинейной волатильности или ставок.
- **Кем используется:** Исследовательские подразделения (Quant Research) крупных инвестиционных банков (JP Morgan, Goldman Sachs) и хедж-фонды, работающие со сложными структурными продуктами.
- **Когда применять в дипломе:**
 - Если размерность задачи $d \geq 10$ (классические сетки умирают).
 - Если задача нелинейная (обычный Монте-Карло не подходит).
 - Если нужно найти не только цену, но и хеджирующую стратегию.
- **Ограничения:** В отличие от метода DGM, Deep BSDE находит решение только для *одного* фиксированного начального условия ($t = 0, X_0$). Если цена актива X_0 изменится, сеть нужно переобучать заново (или использовать методы параметризации X_0 , что сложнее).

3.2 Метод DGM: Глобальное решение и задачи со свободной границей

Статья: *DGM: A deep learning algorithm for solving partial differential equations* [9].

3.2.1 Проблема и мотивация

В то время как метод Deep BSDE решает уравнение вдоль конкретных стохастических траекторий (Lagrangian approach), он имеет существенный недостаток: решение находится только для одной фиксированной точки ($t = 0, X_0$). Если рыночная цена актива изменится, процедуру обучения нужно запускать заново.

Метод DGM (Deep Galerkin Method) предлагает *глобальный* подход (Eulerian approach). Цель метода — обучить нейросеть, которая будет аппроксимировать функцию цены $u(t, x)$ во всей области определения сразу. Это позволяет не только получать мгновенные оценки при изменении цены актива, но и эффективно решать задачи с **свободной границей** (Free Boundary Problems), к которым относится оценка Американских опционов — наиболее распространенного типа деривативов на биржевых рынках.

3.2.2 Математическая теория

Рассмотрим общее параболическое PDE для функции $u(t, x)$ в области $\Omega \subset \mathbb{R}^d$ на интервале $[0, T]$:

$$(\partial_t + \mathcal{L})u(t, x) = 0, \quad (t, x) \in [0, T] \times \Omega, \quad (9)$$

где $\mathcal{L}$ — дифференциальный оператор (для финансов — оператор Блэка-Шоулза или Хестона), включающий вторые производные (Гессиян). Также заданы граничные условия на $\partial\Omega$ и терминальное условие при $t = T$.

Вместо построения сетки, авторы предлагают искать решение в виде нейронной сети $f(t, x; \theta)$. Функция потерь конструируется как взвешенная сумма квадратов невязок (residuals) уравнения в случайно выбранных точках:

$$J(f) = \underbrace{\|\partial_t f + \mathcal{L}f\|_{[0,T] \times \Omega}^2}_{\text{Ошибка внутри области}} + \underbrace{\|f - g\|_{\partial\Omega \times [0,T]}^2}_{\text{Граничные условия}} + \underbrace{\|f(T, \cdot) - \text{Payoff}\|_{\Omega}^2}_{\text{Терминальное условие}} \quad (10)$$

Ключевая математическая инновация: Вычисление оператора $\mathcal{L}f$ требует нахождения вторых производных (матрицы Гессияна). Для нейросети в высокой размерности d это операция сложности $O(d^2)$, что вычислительно запретительно. Авторы предложили метод несмещенной оценки вторых производных с помощью Монте-Карло (аналог Hutchinson's trace estimator), что снижает сложность до $O(d)$ и позволяет работать с размерностями $d = 200$.

3.2.3 Алгоритм и Архитектура

Процесс обучения (Training Loop) выглядит следующим образом:

1. На каждой итерации генерируется пакет (batch) случайных точек внутри области, на границах и во времени T .
2. С помощью автоматического дифференцирования (Autograd) вычисляются производные нейросети по входам $\frac{\partial f}{\partial t}, \frac{\partial f}{\partial x}, \frac{\partial^2 f}{\partial x^2}$.
3. Вычисляется невязка уравнения и обновляются веса θ методом стохастического градиентного спуска (Adam).

Архитектура сети, предложенная авторами, вдохновлена LSTM и ResNet (DGM-Layer). Она включает проброс связей (skip-connections) и мультипликативные взаимодействия, что позволяет избежать затухания градиента при вычислении производных высоких порядков.

3.2.4 Область применения и преимущества

- **Американские опционы:** Это главная "фишка" метода. Задача прайсинга Американского опциона формулируется как задача с препятствием (obstacle problem) или задача со свободной границей. В DGM это реализуется добавлением штрафа в Loss-функцию, который гарантирует, что цена опциона всегда не ниже выплаты при досрочном исполнении. Deep BSDE справляется с этим значительно хуже (требуется Reflected BSDE).
- **Универсальность:** Одна обученная модель дает поверхность цен для любых S_t и t .
- **Сложность вычислений:** Метод является вычислительно тяжелым («expensive»). Взятие вторых производных через граф вычислений нейросети требует большого объема видеопамяти (VRAM) и мощных GPU.
- **Когда применять в дипломе:**
 - Если тема касается **Американских опционов** или Бермудских опционов.
 - Если нужно исследовать зависимость цены от состояния рынка (построить график Value Surface).
 - Если имеется доступ к хорошим вычислительным ресурсам (NVIDIA Tesla T4/P100 и выше).

4 Детальный анализ ключевых исследований

В данном разделе проводится анализ фундаментальных работ, формирующих методологическую базу применения глубокого обучения в задачах финансовой математики. Рассматриваются методы решения уравнений в частных производных (PDE), подходы к хеджированию в условиях рыночных несовершенств, а также алгоритмы калибровки моделей.

4.1 Метод Deep BSDE: Стохастическая оптимизация для высокоразмерных PDE

Статья: *Solving high-dimensional partial differential equations using deep learning* [5].

4.1.1 Проблема и мотивация

Классические сеточные методы (Finite Difference Methods) подвержены «проклятию размерности»: количество узлов сетки растет экспоненциально с увеличением числа активов, что делает их неприменимыми для размерностей $d > 5$. Стандартный метод Монте-Карло, работающий в высоких размерностях, применим преимущественно для линейных PDE. Однако многие современные финансовые задачи (например, оценка CVA, прайсинг с учетом дефолтного риска или различных ставок заимствования) описываются **нелинейными** PDE. Метод Deep BSDE стал первым алгоритмом, позволявшим решать такие уравнения в размерности $d = 100$ за полиномиальное время.

4.1.2 Математическая теория

Метод базируется на теории обратных стохастических дифференциальных уравнений (Backward Stochastic Differential Equations, BSDE). Рассмотрим полулинейное параболическое уравнение для функции цены $u(t, x)$:

$$\frac{\partial u}{\partial t} + \frac{1}{2} \text{Tr}(\sigma \sigma^T \text{Hess}_x u) + \nabla u \cdot \mu + f(t, x, u, \sigma^T \nabla u) = 0, \quad (11)$$

с терминальным условием $u(T, x) = g(x)$.

Согласно нелинейной версии формулы Фейнмана-Каца, решение этого PDE эквивалентно решению системы стохастических уравнений:

$$Y_t = g(X_T) - \int_t^T f(s, X_s, Y_s, Z_s) ds - \int_t^T Z_s^T dW_s, \quad (12)$$

где $Y_t = u(t, X_t)$ — цена дериватива, а $Z_t = \sigma^T \nabla u(t, X_t)$ — вектор хеджирования.

4.1.3 Алгоритм и применимость

Идея метода заключается в аппроксимации неизвестной функции градиента Z_t нейронной сетью на каждом временном шаге дискретизации Эйлера-Маруямы. Функция потерь строится на невязке терминального условия:

$$L(\theta) = \mathbb{E} [|Y_T(\theta) - g(X_T)|^2]. \quad (13)$$

Данный подход является стандартом (SOTA) для задач высокой размерности (например, оценка корзинных опционов). Его применение целесообразно в случаях, когда требуется найти не только цену, но и стратегию хеджирования в условиях нелинейной динамики.

4.2 DGM: Бессеточный метод Галёркина

Статья: *DGM: A deep learning algorithm for solving partial differential equations* [9].

4.2.1 Проблема и мотивация

Метод Deep BSDE решает уравнение вдоль конкретных стохастических траекторий, что ограничивает решение одной фиксированной точкой ($t = 0, X_0$). Метод DGM (Deep Galerkin Method) предлагает глобальный подход, обучая нейросеть аппроксимировать функцию цены во всей области определения. Это позволяет эффективно решать задачи со свободной границей (Free Boundary Problems), к которым относится оценка Американских опционов.

4.2.2 Математическая теория

Обучение сводится к минимизации квадрата невязки дифференциального оператора $\mathcal{L}$ в случайно выбранных точках области Ω :

$$J(f) = \|\partial_t f + \mathcal{L}f\|_{\Omega}^2 + \|f - g\|_{\partial\Omega}^2 + \|f(T, \cdot) - \text{Payoff}\|^2. \quad (14)$$

Авторы предложили метод несмещенной оценки вторых производных (Гессиана) с помощью Монте-Карло, что снижает вычислительную сложность с $O(d^2)$ до $O(d)$.

4.2.3 Алгоритм и применимость

Архитектура сети, основанная на DGM-слоях (модификация ResNet/LSTM), позволяет избежать затухания градиента при вычислении производных высоких порядков. Метод наиболее эффективен для задач, требующих построения полной поверхности цен (Value Surface) и оценки опционов с возможностью досрочного исполнения, однако требует значительных вычислительных ресурсов (GPU) для обучения.

4.3 Deep Hedging: Оптимизация торговых стратегий при наличии трений

Статья: *Deep Hedging* [3].

4.3.1 Проблема и мотивация

Классическая теория (Блэк-Шоулз) предполагает отсутствие транзакционных издержек и возможность непрерывной торговли. В реальных условиях эти допущения нарушаются, делая классическое дельта-хеджирование субоптимальным. Подход *Deep Hedging* предлагает смену парадигмы: от поиска «теоретической цены» к поиску оптимальной стратегии, минимизирующей риск с учетом рыночных трений.

4.3.2 Математическая теория

Задача формулируется как проблема стохастического управления. Итоговый капитал PL_T зависит от стратегии δ и транзакционных издержек C_k , которые зависят от оборота $|\delta_k - \delta_{k-1}|$:

$$PL_T(\delta) = -Z + p_0 + \sum (\delta \cdot \Delta S) - \sum C_k(\delta_k, \delta_{k-1}). \quad (15)$$

Целью является минимизация выпуклой меры риска ρ (например, CVaR или экспоненциальной полезности):

$$\pi(Z) = \inf_{\delta \in \mathcal{H}} \rho(-PL_T(\delta)). \quad (16)$$

4.3.3 Алгоритм и применимость

Поскольку издержки зависят от предыдущего состояния портфеля, задача является немарковской относительно цены актива. Для решения используются рекуррентные нейронные сети (RNN/LSTM). Метод демонстрирует высокую практическую ценность, позволяя моделировать стратегии, превосходящие классические подходы по показателю риск/доходность на рынках с комиссиями.

4.4 Deep Calibration: Ускорение калибровки сложных моделей

Статья: *Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models* [6].

4.4.1 Проблема и мотивация

Современные модели, такие как *Rough Volatility*, точно описывают динамику рынка, но не имеют аналитических решений. Их калибровка традиционными методами требует множественных симуляций Монте-Карло, что занимает неприемлемо много времени для операционной деятельности.

4.4.2 Суть метода

Предлагается двухэтапный подход:

1. **Офлайн-обучение:** Нейросеть обучается на синтетических данных аппроксимировать функцию цены $F(\theta) \approx P(\theta)$. Сеть выступает в роли детерминированного суррогата стохастической модели.
2. **Онлайн-калибровка:** При поступлении рыночных данных параметры модели θ^* находятся путем минимизации разницы между предсказанием сети и рынком с использованием градиентного оптимизатора.

4.4.3 Алгоритм и применимость

Использование суррогатной модели позволяет ускорить процесс калибровки в 10 000–60 000 раз. Метод применим к широкому классу моделей (Хестон, Rough Bergomi) и является эффективным решением проблемы «вычислительного бутылочного горлышка» в финансовом моделировании.

4.5 CaNN: Калибровка через механизм обратного распространения ошибки

Статья: *A neural network-based framework for financial model calibration* [8].

4.5.1 Суть метода

В отличие от подхода с внешним оптимизатором, фреймворк CaNN (Calibration Neural Network) использует механизм обратного распространения ошибки (Backpropagation) для решения обратной задачи. Процесс разделен на две фазы: обучение сети (forward pass) и калибровка (backward pass). На этапе калибровки веса сети замораживаются, а входной слой, соответствующий параметрам модели, объявляется обучаемым. Градиент ошибки между предсказанной и рыночной ценой распространяется до входа, обновляя параметры модели.

4.5.2 Применимость

Данный подход позволяет выполнять калибровку, используя встроенные возможности фреймворков глубокого обучения (например, PyTorch) без подключения сторонних солверов, что упрощает архитектуру программного решения.

4.6 Теоретическое обоснование: Рандомизированные сигнатуры

Статья: *Discrete-time signatures and randomness in reservoir computing* [4].

4.6.1 Суть метода

Статья предоставляет фундаментальное теоретическое обоснование эффективности рекуррентных нейронных сетей (RNN) в задачах прогнозирования финансовых временных рядов. Авторы связывают RNN с теорией сигнатур путей (Path Signatures) и рядами Вольтерра, доказывая свойство универсальной аппроксимации для фильтров с затухающей памятью.

4.6.2 Применимость

Работа обосновывает выбор архитектур типа LSTM/GRU для задач, зависящих от траектории (path-dependent), и предлагает альтернативные методы на основе Reservoir Computing, которые могут быть вычислительно эффективнее полного обучения глубоких сетей.

4.7 Инженерные аспекты: Обеспечение финансовой корректности

Статья: *Deep learning calibration of option pricing models: some pitfalls and solutions* [7].

4.7.1 Проблема и решения

Прямое применение стандартных методов ML может приводить к моделям, допускающим статический арбитраж или имеющим разрывные производные (Греки). Автор предлагает ряд инженерных решений:

1. Использование гладких функций активации класса C^2 (Softplus, ELU) вместо ReLU для корректного вычисления вторых производных (Гамма).
2. Введение штрафов (Soft Constraints) в функцию потерь за нарушение условий отсутствия арбитража (например, нарушение монотонности цены по времени или выпуклости по страйку).

4.7.2 Применимость

Данные рекомендации являются критически важными при практической реализации моделей для обеспечения их робастности и соответствия финансовой теории.

Таблица 2: Сравнение нейросетевых подходов

Метод	Задача	Сложность	Ресурсы (GPU)
Deep BSDE	Прайсинг, $d \sim 100$	Средняя	Средние (T4)
DGM	Американские опционы	Высокая	Высокие (V100/A100)
Deep Hedging	Хеджирование с тренингами	Средняя	Средние/Высокие
Deep Calibration	Мгновенная калибровка	Низкая	Низкие (любой GPU)

5 Сравнительный анализ и Выводы

5.1 План практического исследования

На основе проведенного обзора, для практической части дипломной работы выбран метод **Deep Calibration**. Данный подход позволяет решить актуальную проблему медленной калибровки, демонстрируя превосходство над классическими методами. Для реализации будет использован фреймворк PyTorch, а обучение будет проводиться с использованием облачных ресурсов (Google Colab/Kaggle) для ускорения вычислений.

Список литературы

- [1] Christian Beck, Weinan E и Arnulf Jentzen. “Machine learning approximation algorithms for high-dimensional fully nonlinear partial differential equations and second-order backward stochastic differential equations”. В: *Journal of Nonlinear Science* 29.4 (2019), с. 1563—1619.
- [2] Fischer Black и Myron Scholes. “The pricing of options and corporate liabilities”. В: *Journal of political economy* 81.3 (1973), с. 637—654.
- [3] Hans Buehler и др. “Deep hedging”. В: *Quantitative Finance* 19.8 (2019), с. 1271—1291.
- [4] Christa Cuchiero и др. “Discrete-time signatures and randomness in reservoir computing”. В: *IEEE Transactions on Neural Networks and Learning Systems* 32.9 (2021), с. 3904—3918.
- [5] Jiequn Han, Arnulf Jentzen и Weinan E. “Solving high-dimensional partial differential equations using deep learning”. В: *Proceedings of the National Academy of Sciences* 115.34 (2018), с. 8505—8510.
- [6] Blanka Horvath, Aitor Muguruza и Mehdi Tomas. “Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models”. В: *Quantitative Finance* 21.1 (2021). Preprint 2019, с. 11—27.
- [7] Andrey Itkin. “Deep learning calibration of option pricing models: some pitfalls and solutions”. В: *arXiv preprint arXiv:1906.03507* (2019).
- [8] Shuaiqiang Liu и др. “A neural network-based framework for financial model calibration”. В: *Journal of Mathematics in Industry* 9.1 (2019), с. 1—28.
- [9] Justin Sirignano и Konstantinos Spiliopoulos. “DGM: A deep learning algorithm for solving partial differential equations”. В: *Journal of Computational Physics* 375 (2018), с. 1339—1364.